

Ακαδημαϊκός Βοηθός με Agentic RAG

Deadline: 25/5/2026, ΩΡΑ: 23:59

1. Περιγραφή Εργασίας

Στην εργασία αυτή καλείστε να σχεδιάσετε και να υλοποιήσετε έναν **Ακαδημαϊκό Βοηθό**. Πρόκειται για ένα σύστημα που δέχεται ερωτήσεις σε φυσική γλώσσα και απαντάει αξιοποιώντας ένα corpus ακαδημαϊκών εγγράφων που θα συλλέξετε εσείς οι ίδιοι. Το σύστημα συνδυάζει τεχνικές Retrieval-Augmented Generation (RAG) με Agentic AI, δηλαδή πολλαπλούς agents που συνεργάζονται, παίρνουν αποφάσεις και εκτελούν επαναληπτικά loops.

Το σύστημα πρέπει να τρέχει τοπικά σε κανονικό PC, χρησιμοποιώντας αποκλειστικά open-source και δωρεάν εργαλεία, χωρίς APIs που απαιτούν πληρωμή.

Κάθε ομάδα επιλέγει μία θεματική περιοχή και συλλέγει μόνη της το corpus εγγράφων που θα χρησιμοποιήσει. Τα έγγραφα πρέπει να αντληθούν μέσω δημόσιου API, ώστε η συλλογή να είναι αυτοματοποιημένη και αναπαραγώγιμη. Ανάλογα με το API, μπορείτε να χρησιμοποιήσετε paper abstracts ή πλήρη papers, αν το επιτρέπει η υπηρεσία. Το corpus πρέπει να περιλαμβάνει τουλάχιστον 50 έγγραφα.

Προτεινόμενες θεματικές περιοχές και αντίστοιχα APIs:

Θεματική ενότητα	API	Περιεχόμενο
Artificial intelligence	Semantic scholar api, arxiv api	Abstracts και πλήρη papers (arxiv)
Biomedical applications	Pubmed api (e-utilities)	Abstracts ιατρικών δημοσιεύσεων
Bioengineering	Pubmed api, europe pmc api	Abstracts βιοϊατρικής μηχανικής
Economics	Ssrn api, repec/ideas api	Abstracts οικονομικών ερευνών
Defense	Semantic scholar api, dtic (open)	Abstracts αμυντικής έρευνας

Μπορείτε να επιλέξετε και άλλη θεματική, αρκεί να χρησιμοποιείτε δημόσιο και δωρεάν API για τη συλλογή των εγγράφων. Στο report πρέπει να τεκμηριώσετε την επιλογή σας και να περιγράψετε τη διαδικασία συλλογής.

2. Αρχιτεκτονική Συστήματος

Γενική εικόνα

Το σύστημα αποτελείται από δύο επίπεδα. Το πρώτο είναι το **RAG pipeline**, δηλαδή η υποδομή που φορτώνει τα έγγραφα, τα σπάει σε chunks, δημιουργεί embeddings και τα αποθηκεύει σε ένα vector store ώστε να μπορούν να αναζητηθούν αργότερα. Το δεύτερο είναι το **agentic layer**, δηλαδή τρεις (τουλάχιστον) agents που χρησιμοποιούν το RAG pipeline ως εργαλείο και συντονίζονται μεταξύ τους για την παραγωγή του αποτελέσματος

Σημαντική διευκρίνιση: **όλοι οι agents χρησιμοποιούν το ίδιο LLM**. Δεν χρειάζεστε τρία ξεχωριστά μοντέλα. Φορτώνετε ένα μοντέλο μία φορά μέσω Ollama και κάθε agent το καλεί με διαφορετικό system prompt.

RAG Pipeline

Πριν μπορέσουν οι agents να λειτουργήσουν, πρέπει να έχετε στήσει τη βάση δεδομένων εγγράφων. Αυτό γίνεται σε τέσσερα βήματα. Πρώτα, φορτώνετε τα έγγραφα του corpus (PDFs ή text). Μετά, τα χωρίζετε σε μικρότερα κομμάτια (chunks) κατάλληλου μεγέθους, τυπικά 500–1000 tokens, με overlap 50–150 tokens ώστε να μη χαθεί πληροφορία στα σύνορα. Στη συνέχεια, μετατρέπετε κάθε chunk σε ένα vector (embedding) χρησιμοποιώντας ένα embedding model. Τέλος, αποθηκεύετε τα vectors σε ένα vector store (π.χ. ChromaDB) μαζί με metadata (τίτλος εγγράφου, συγγραφείς, έτος). Αυτή η διαδικασία γίνεται μία φορά (offline). Κατά τη λειτουργία, η ερώτηση του χρήστη μετατρέπεται κι αυτή σε vector και αναζητούνται τα πιο παρόμοια chunks.

AI Agents

Πάνω στο RAG pipeline λειτουργούν οι agents. Ο πρώτος agent (**Query Analyzer**) δέχεται την ερώτηση του χρήστη και αποφασίζει πώς θα τη χειριστεί. Αν είναι αρκετά συγκεκριμένη, την προωθεί απευθείας. Αν είναι σύνθετη, τη σπάει σε υπο-ερωτήματα. Για παράδειγμα, η ερώτηση «Σύγκρινε τους Transformers με τα RNNs ως προς την παραλληλοποίηση» γίνεται δύο ξεχωριστά queries: ένα για Transformers, ένα για RNNs. Αν η ερώτηση είναι εκτός πεδίου, ενημερώνει τον χρήστη.

Ο δεύτερος agent (**Retriever & Evaluator**) εκτελεί vector search στη από την βάση, φέρνει τα πιο σχετικά chunks και τα αξιολογεί αν απαντούν πράγματι στο ερώτημα; Αν όχι, διαμορφώνει εκ νέου το query (reformulation) και ξαναψάχνει. Αυτό το loop αποτελεί τον πυρήνα του agentic σχεδιασμού: ο agent σκέφτεται (reason), ενεργεί (act), παρατηρεί τα αποτελέσματα (observe) και αποφασίζει αν τελείωσε ή πρέπει να ξαναπροσπαθήσει (decide). Μετά από μέγιστο αριθμό

προσπαθειών (π.χ. 3), σταματάει και αναφέρει ότι δεν βρέθηκε επαρκής πληροφορία. Ο agent πρέπει να παίρνει πραγματικές αποφάσεις, δεν αρκεί ένα απλό passthrough.

Ο τρίτος agent (**Synthesizer**) παίρνει τα validated chunks και συνθέτει μια σαφή απάντηση σε φυσική γλώσσα. Προαιρετικά κάθε ισχυρισμός πρέπει να συνοδεύεται από inline citation με αναφορά στο έγγραφο προέλευσης, π.χ. [Document: attention_is_all_you_need, 2017]. Αν η πληροφορία είναι ελλιπής, ο agent πρέπει να το δηλώνει ρητά αντί να επινοεί απάντηση (hallucinate).

Ροή εκτέλεσης

Η ροή εκτέλεσης σε μορφή γραφήματος είναι η εξής. Η ερώτηση του χρήστη πηγαίνει στον Query Analyzer, ο οποίος είτε παράγει ένα ή περισσότερα queries είτε απορρίπτει την ερώτηση ως εκτός πεδίου. Τα queries πηγαίνουν στον Retriever & Evaluator, ο οποίος αναζητά chunks, τα αξιολογεί, και αν δεν είναι ικανοποιητικά κάνει reformulation και ξαναψάχνει (conditional loop). Τα validated chunks πηγαίνουν στον Synthesizer, ο οποίος παράγει την τελική απάντηση με citations. Στο LangGraph, κάθε agent είναι ένα node και οι μεταβάσεις είναι conditional edges. Το retry loop του Retriever υλοποιείται ως ένα conditional edge που ελέγχει αν τα αποτελέσματα είναι ικανοποιητικά.

Οδηγίες Υλοποίησης

Ξεκινήστε στήνοντας σωστά το περιβάλλον εργασίας. Εγκαταστήστε το Ollama, κατεβάστε ένα κατάλληλο μοντέλο, για παράδειγμα με την εντολή `ollama pull mistral`, και εγκαταστήστε όλα τα απαραίτητα Python dependencies όπως `langchain`, `langgraph`, `chromadb` και ό,τι άλλο απαιτεί το project σας. Πριν προχωρήσετε, βεβαιωθείτε ότι μπορείτε να εκτελέσετε inference τοπικά και ότι το μοντέλο αποκρίνεται σωστά σε δοκιμαστικά prompts.

Στη συνέχεια, συλλέξτε το corpus σας μέσω του API που έχετε επιλέξει και προχωρήστε στη διαμόρφωση του RAG pipeline. Φορτώστε τα έγγραφα, εφαρμόστε chunking και πειραματιστείτε με το μέγεθος των τμημάτων. Μια ενδεικτική περιοχή είναι 500 έως 1000 tokens, με overlap 50 έως 150 tokens, ώστε να διατηρείται το συμφραζόμενο. Δημιουργήστε embeddings για κάθε chunk και αποθηκεύστε τα σε ένα vector store. Φροντίστε κάθε chunk να συνοδεύεται από metadata όπως τίτλος εγγράφου, συγγραφείς και έτος δημοσίευσης, ώστε να είναι δυνατή η ιχνηλασιμότητα των απαντήσεων. Έπειτα, υλοποιήστε τους agents ως ξεχωριστά nodes στο LangGraph. Ορίστε με σαφήνεια ποιο state μοιράζονται μεταξύ τους και σχεδιάστε τα conditional edges που θα καθορίζουν τη ροή εκτέλεσης, για παράδειγμα λογική επανάληψης σε περίπτωση αποτυχίας του retriever.

Τέλος, αξιολογήστε το σύστημα με τουλάχιστον δέκα ερωτήσεις που θα διαμορφώσετε εσείς. Οι ερωτήσεις θα πρέπει να καλύπτουν τρεις κατηγορίες: απλές, όπου επαρκεί μία αναζήτηση, σύνθετες, που απαιτούν διάσπαση του ερωτήματος ή σύνθεση πληροφορίας από πολλαπλές πηγές, και εκτός πεδίου, όπου δεν υπάρχει διαθέσιμη απάντηση στο corpus. Καταγράψτε συστηματικά τα αποτελέσματα και τα πιθανά σφάλματα.

Προς διευκόλυνση της τεχνικής υλοποίησης μπορείτε να χρησιμοποιήσετε το εργαλείο n8n για την αυτοματοποίηση των διαδικασιών

Παραδοτέα

Παραδίδετε τον πλήρη κώδικα ως GitHub repository ή ως συμπιεσμένο αρχείο zip. Το πακέτο παράδοσης πρέπει να περιλαμβάνει README.md με σαφείς οδηγίες εγκατάστασης και εκτέλεσης, καθώς και αρχείο requirements.txt. Το σύστημα οφείλει να εκτελείται χωρίς ιδιωτικά API keys, δηλαδή να μην εξαρτάται από κλειδιά που δεν μπορούν να διανεμηθούν.

Επιπλέον, παραδίδετε report σε μορφή PDF, έκτασης 5 έως 10 σελίδων. Το report πρέπει να περιλαμβάνει αρχιτεκτονικό διάγραμμα, τεκμηρίωση των βασικών σχεδιαστικών επιλογών (π.χ. επιλογή μοντέλου, επιλογή chunk size και overlap), αναλυτική περιγραφή κάθε agent μαζί με τα prompts που χρησιμοποιεί, περιγραφή της διαδικασίας συλλογής και οργάνωσης του corpus, καθώς και αποτελέσματα αξιολόγησης πάνω στις test ερωτήσεις με ενδεικτικά παραδείγματα σωστών και λανθασμένων απαντήσεων. Καταγράψτε επίσης τις κύριες δυσκολίες που αντιμετωπίσατε και πώς τις διαχειριστήκατε.

Τέλος, παραδίδετε ένα σύντομο demo διάρκειας 2 έως 3 λεπτών, είτε ως screencast είτε ως live επίδειξη. Το demo πρέπει να δείχνει τρία σενάρια: μία ερώτηση που απαντιέται σωστά, μία σύνθετη ερώτηση που ενεργοποιεί query decomposition, και μία ερώτηση που ενεργοποιεί το retry loop.